

HEALTHCARE AI COGNITIVE CORE

Comprehensive System Architecture, Custom Vector RAG Pipeline,
and Gemma 3 (4B) Local LLM Integration Documentation

Project Info & Metadata

Generated Date: June 12, 2026

Engine Version: 1.0.0 (Custom SQLite Vector Store)

AI Models: Gemma 3 (4B) Instruction, nomic-embed-text (137M)

Stack: Next.js (Frontend) & FastAPI (Backend) & SQLite (Database)

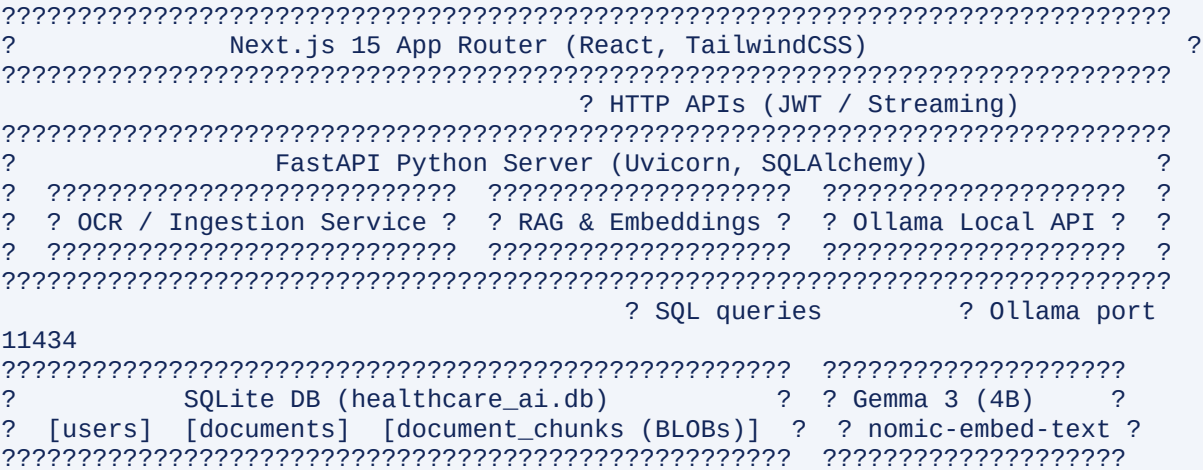
1. Executive Summary

The AI Healthcare Application is a secure, privacy-preserving client-server platform designed for medical professionals and patients to analyze healthcare insurance policies and medical documentation. At its core, the system runs a local RAG (Retrieval-Augmented Generation) engine that combines document OCR/extraction, chunking, high-dimensional vector embeddings, and a local Gemma 3 (4B) LLM to answer complex user queries, extract hidden risks (co-pays, waiting periods, room-rent limits), and set proactive policy reminders. By hosting everything locally via Ollama and SQLite, patient data privacy is strictly enforced without leakage to external cloud services.

2. Core System Architecture & Data Flow

1. INGEST & EXTRACT	User uploads policy PDF; backend extracts raw text using PyMuPDF/OCR.
2. CHUNK & CLEAN	Text is cleaned and split into semantic chunks (500 tokens with 10% overlap).
3. GENERATE VECTORS	Each chunk is embedded into a 768-dimensional vector via nomic-embed-text.
4. SQLITE STORAGE	Chunks are stored in SQLite; vectors are stored as binary float arrays (BLOBs).
5. SEMANTIC QUERY	User asks a question. Query is embedded; cosine similarity finds matching chunks.
6. LOCAL LLM INFERENCE	Retrieved chunks + query + system prompt are fed into Gemma 3 (4B) via Ollama.
7. UI RENDERING	Gemma 3's response streams back to the Next.js React frontend via SSE/FastAPI.

Architecture Component Mapping



3. Complete Technology Stack

Layer	Technology	Purpose & Description
Frontend Core	Next.js 15 (React)	Standard application layout, routing, dynamic views, components.
Frontend Styles	Vanilla CSS	Theme configuration, custom glassmorphism components, layouts.
State / Context	React Context API	User authentication session management and theme toggles.
Backend Server	FastAPI (Python)	High-performance async API server with automatic OpenAPI docs.
WebServer	Uvicorn	ASGI web server running the FastAPI python service on port 8000.
Database ORM	SQLAlchemy + aiosqlite	Asynchronous SQLite interaction and Object-Relational Mapping.
Database Engine	SQLite (Local File)	Zero-configuration persistent file storing relational and vector data.
Vector Search	Custom Numpy/SQLite	Calculates cosine-similarity directly in the python/SQL layer.
Local LLM Core	Ollama (Gemma 3 4B)	Ultra-fast response generation using 4.7B parameter Google Gemma 3.
Embedding Core	Ollama (nomic-embed-text)	Transforms text chunks into dense 768-dimension vector spaces.
Document Extract	PyMuPDF (fitz)	Parses, cleans, and structures text from uploaded PDF policy files.
Authentication	JWT (JSON Web Tokens)	Secure, stateless user sessions with password hashing via bcrypt.

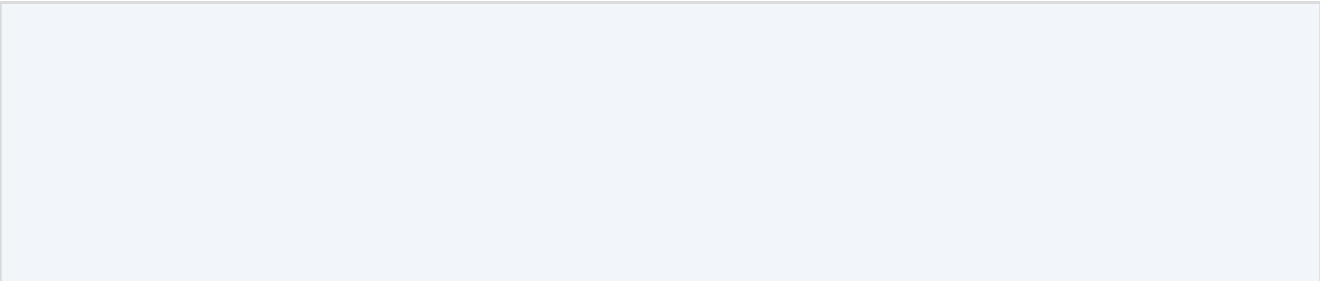
4. Component Features

Interactive Chat Interface Provides a conversational thread where the user can query their insurance documents. Supports streaming tokens directly for a highly responsive, zero-delay UI.
Automated Compliance & Risk Analysis Extracts critical elements from medical insurance paperwork (like co-payments, waiting periods, room-rent limits) and flags high-severity exceptions automatically.
Policy Reminders & Due Dates Analyzes documents for renewal deadlines and premium payment schedules, storing them in the SQLite DB to notify users in the dashboard.
Custom Local RAG Pipeline Bypasses slow cloud databases. Uses a custom SQLite integration storing floats as binary BLOBs and performing fast vector math locally to fetch context in milliseconds.

5. Custom SQLite Vector DB & RAG Pipeline

Unlike enterprise architectures that rely on expensive external Vector Databases (like Pinecone or Milvus), this project employs a custom SQLite-based vector storage system built for speed, simplicity, and zero external dependencies. Let's explore how it's implemented.

5.1 Database Schema (document_chunks)



5.2 How the Vector Search Operations Work

- **Chunking:**
- **Embedding:**
- **BLOB Storage:**
- **Cosine Similarity:**
- **Prompt Synthesis:** The matching context chunks are combined and injected into the LLM system prompt as context. Gemma 3 uses this context to synthesize a fact-grounded response, eliminating hallucinations.

5.3 Gemma 3 (4B) Local LLM Advantage

The project integrates Gemma 3 (4B) Instruction model. Gemma 3 represents a significant upgrade over previous generation models like Llama 3.2 (3B). It offers:

1. **Faster Inference Speed:** Highly optimized attention mechanisms make it generate responses at higher tokens per second on consumer hardware.
2. **Improved Context Utilization:** Better multi-turn comprehension and strict adherence to structured JSON schemas, which is crucial for parsing policy details.
3. **Privacy:** Completely offline execution ensures full compliance with medical privacy laws (HIPAA/GDPR).

RAG CONTEXT EXTRACTION & QUERY LIFECYCLE

User Query ??? [Embed Query] ??? [Cosine Similarity vs SQLite BLOBs] ???
?
User Screen ??? [Stream Tokens] ??? [Ollama: Gemma 3] ??? [Synthesized Context + Prompt]